# Chain-of-Thought (`Reasoning:`) Mechanics Across Post-Training Algorithms

## How the `Reasoning:` Trace Is Distilled, Formatted, Optimized, and Reinforced in SFT, DPO, ORPO, and GRPO

| | |
|---|---|
| **Project** | **AskBeforeAnswer** (Clarification-Seeking Qwen 2.5 7B Post-Training Pipeline) |
| Scope | Data Preprocessing (`preprocess.py`), Model Training (`trainer.py`), Rewards (`criteria.py`) |
| **Algorithms Covered** | Supervised Fine-Tuning (SFT), Direct Preference Optimization (DPO), Odds Ratio Preference Optimization (ORPO), Group Relative Policy Optimization (GRPO) |
| Audience | Machine Learning Engineers, Applied Researchers, and Codebase Contributors |

## 1. Motivation: Why Explicit Reasoning Traces Matter for Clarification

When an open-domain Large Language Model (LLM) is presented with an underspecified user prompt (e.g., *"Who won the US Open?"* or *"How do I make pasta?"*), standard instruction-tuned models default to guessing a single arbitrary interpretation and answering immediately. Teaching a 7B-parameter language model to reliably distinguish between **unambiguous queries** (which should be answered directly) and **ambiguous queries** (which require targeted clarification) demands more than binary classification.

In the **AskBeforeAnswer** architecture, every model completion is constrained at the token level to emit a structured four-field schema:

```
Action: Clarify | Answer
Reasoning: <Chain-of-Thought rationale explaining why the query is ambiguous or clear>
Facets: [<JSON list of missing informational attributes if Clarify, else empty list []>]
Response: <Targeted clarifying question grounded in the facets OR direct factual answer>
```

### 1.1 The Three Architectural Functions of the `Reasoning:` Field

Rather than treating the `Reasoning:` line as decorative text, the pipeline relies on it to serve three concrete machine learning functions across the lifecycle of the model:

1. **Autoregressive Conditioning Bridge:** In causal (left-to-right) transformer decoding, each generated token conditions all subsequent tokens via causal self-attention. Because `Reasoning:` sits on Line 2 (immediately after `Action:` and prior to `Facets:` and `Response:`), generating the Chain-of-Thought (CoT) rationale decomposes the specific dimensions of ambiguity (e.g., missing temporal, entity, or geographic qualifiers) *before* the model populates the `Facets:` list and synthesizes the final `Response:`.
2. **Symmetric Contrastive Signal Against Formatting Shortcuts:** In preference optimization algorithms (DPO and ORPO), if rejected responses are merely malformed or truncated strings, the policy learns a superficial formatting shortcut rather than genuine ambiguity reasoning. By pairing every gold completion with an adversarial hard-negative completion that shares the exact same four-field structure—but contains

a *logically flawed* Reasoning: trace—preference optimization is forced to assign credit and blame to the semantic validity of the reasoning chain itself.

3. **On-Policy Test-Time Compute Buffer in Reinforcement Learning:** During online reinforcement learning (GRPO), the model is no longer given pre-written rationales; it samples its own Reasoning: tokens on-policy prior to generating Facets: and Response:. Rollouts that articulate clearer on-policy reasoning chains produce higher-quality downstream facets and answers, earning higher trajectory rewards that reinforce those reasoning tokens via policy gradients.

**Autoregressive Ordering Note: Action → Reasoning → Facets → Response**

Because Action: appears on Line 1 and Reasoning: appears on Line 2, causal self-attention during inference means the binary routing token (Clarify vs. Answer) is predicted directly from the prompt $x$, while the Reasoning: trace conditions the extraction of Facets: (Line 3) and the phrasing of Response: (Line 4). During backpropagation in training, however, gradients from Reasoning:, Facets:, and Response: flow backward through the entire network's shared parameters $\theta$.

## 2. Origins of the Reasoning: Trace in Data Preprocessing

The raw source benchmark (sewon/ambig_qa) provides only natural language questions and ground-truth question-answer pairs; it contains no native Chain-of-Thought rationales. During the DVC preprocess stage (src/ask_before_answer/data/preprocess.py), the SyntheticGenerator class uses unsloth/qwen2.5-7b-instruct-unsloth-bnb-4b to synthesize two distinct categories of reasoning traces:

1. **Positive Gold CoT (reasoning):** Generated via generate_batch() using SYSTEM_PROMPT. For ambiguous queries, it explains why multiple interpretations exist; for unambiguous queries, it explains why the intent is fully specified. This field populates both the SFT target (sft_train.jsonl) and the chosen ($y_w$) completion in the preference dataset (dpo_train.jsonl).
2. **Adversarial Hard-Negative CoT (neg_reasoning):** Generated via generate_negative_batch() using NEGATIVE_SYSTEM_PROMPT. The instructor model is explicitly prompted to write a *plausible-sounding but logically incorrect* reasoning chain that rationalizes the opposite, wrong action (e.g., arguing that a genuinely ambiguous question is crystal clear to justify a premature Action: Answer, or inventing spurious ambiguity on a clear question to justify an unnecessary Action: Clarify). This field populates the rejected ($y_l$) completion in dpo_train.jsonl.

## 3. Training Mechanics Across the Four Supported Algorithms

To understand how the Reasoning: tag participates in model optimization, it is essential to distinguish between the **input prompt** ($x$), the **target completion** ($y$), the **loss function** ($\mathcal{L}(\theta)$), and the resulting **parameter gradients** ($\nabla_\theta \mathcal{L}$) in src/ask_before_answer/training/trainer.py.

### 3.1 1. Supervised Fine-Tuning (SFT) — run_sft_training()

#### Input ($x$) vs. Target ($y$) Boundary

In run_sft_training() (trainer.py, lines 220–237), each record from sft_train.jsonl is formatted into a three-message ChatML conversation:

- **Conditioning Input Prompt ($x$):** The system message (which contains only the format specification template "Reasoning: <your reasoning>\n") and the user message (example["input"], i.e., the raw question).

- **Target Output Sequence ($y$):** The `assistant` message, which concatenates the four ground-truth fields into a single target completion string:

```
Action: {example['output']['action']}
Reasoning: {example['output']['reasoning']}
Facets: {example['output']['facets']}
Response: {example['output']['response']}
```

### Loss Function & Dual Role Under Autoregressive Teacher Forcing

SFT minimizes the standard token-level negative log-likelihood (Cross-Entropy loss) over the target completion tokens $y = (y_1, y_2, \ldots, y_{|y|})$ conditioned on the prompt $x$:

**SFT Cross-Entropy Objective Decomposed by Schema Field**

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}) = -\left( \sum_{t \in \mathcal{T}_{\text{Action}}} + \underbrace{\sum_{t \in \mathcal{T}_{\text{Reasoning}}}}_{\text{Direct CE on CoT}} + \sum_{t \in \mathcal{T}_{\text{Facets}}} + \sum_{t \in \mathcal{T}_{\text{Response}}} \right) \log \pi_\theta(y_t \mid x, y_{<t}) \tag{1}$$

Because causal language modeling uses **teacher forcing** during the forward pass (where the ground-truth sequence is shifted by one token position to serve simultaneously as input context and target labels), every token inside `Reasoning: {reasoning}` performs two distinct operations during each training step:

1. **As a Supervised Target Label ($y_t$):** For every token index $t \in \mathcal{T}_{\text{Reasoning}}$, the token is the ground-truth label in the Cross-Entropy loss $\log \pi_\theta(y_t \mid x, y_{<t})$. Backpropagation computes direct gradients $\nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})$ that train the model's weights to generate both the literal header `"Reasoning:"` and the substantive Chain-of-Thought rationale.
2. **As Teacher-Forced Input Context ($y_{<t}$) for Downstream Fields:** When the transformer predicts tokens at positions $t \in \mathcal{T}_{\text{Facets}} \cup \mathcal{T}_{\text{Response}}$, the ground-truth `Reasoning:` tokens lie inside the conditioning prefix $y_{<t}$. The attention heads attend directly to the teacher-forced `Reasoning:` tokens to predict the facets and final response.

## 3.2 2. Direct Preference Optimization (DPO) — `run_dpo_training()`

### Input ($x$) vs. Contrastive Targets ($y_w, y_l$)

In `run_dpo_training()` (`trainer.py`, lines 293–392), the trainer loads paired preference records from `dpo_train.jsonl` constructed by `prepare_dpo_dataset()` (`preprocess.py`, lines 452–520):

- **Input Prompt ($x$):** The ChatML-formatted `system` instruction and `user` question, ending with the assistant generation prompt header. It contains *no* reasoning trace.
- **Chosen Target Completion ($y_w$):** The 4-line completion containing the gold action, the **valid CoT trace** (`Reasoning: {reasoning}`), the valid facets, and the gold response.
- **Rejected Target Completion ($y_l$):** The 4-line completion containing the opposite action, the **adversarial hard-negative CoT trace** (`Reasoning: {neg_reasoning}`), and the wrong response.

### How DPO Shifts Log-Probabilities & Computes Gradients Over `Reasoning:`

In DPO, both $y_w$ and $y_l$ are evaluated as **target output sequences** ($y$-values). During a training step, `DPOTrainer` runs teacher-forced forward passes on $(x, y_w)$ and $(x, y_l)$ through both the trainable policy $\pi_\theta$ and

the frozen SFT reference policy $\pi_{\text{ref}}$. Crucially, prompt tokens $x$ are masked out; the sequence log-probability $\log \pi_\theta(y \mid x)$ is computed by summing the next-token log-probabilities across all target completion tokens:

$$\begin{aligned}\log \pi_\theta(y \mid x) &= \sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}) \\ &= \sum_{t \in \mathcal{T}_{\text{Action}}} \log \pi_\theta(y_t \mid \dots) + \underbrace{\sum_{t \in \mathcal{T}_{\text{Reasoning}}} \log \pi_\theta(y_t \mid x, y_{<t})}_{\text{typically 50–70\% of all tokens in } y} + \sum_{t \in \mathcal{T}_{\text{Facets}} \cup \mathcal{T}_{\text{Response}}} \log \pi_\theta(y_t \mid \dots)\end{aligned} \tag{2}$$

These summed target log-probabilities are substituted into the Bradley-Terry DPO loss:

**DPO Objective & Gradient Flow Over Chosen ($y_w$) and Rejected ($y_l$) Targets**

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)} \left[ \log \sigma \left( \beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right] \tag{3}$$

Taking the gradient with respect to $\theta$ yields:

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\theta) = -\beta\, \mathbb{E}_{(x, y_w, y_l)} \left[ w(x, y_w, y_l) \left( \underbrace{\sum_{t=1}^{|y_w|} \nabla_\theta \log \pi_\theta(y_{w,t} \mid x, y_{w,<t})}_{\text{Pulls UP valid } \texttt{Reasoning:} \text{ tokens}} - \underbrace{\sum_{t=1}^{|y_l|} \nabla_\theta \log \pi_\theta(y_{l,t} \mid x, y_{l,<t})}_{\text{Pushes DOWN } \texttt{neg_reasoning} \text{ tokens}} \right) \right] \tag{4}$$

where $w(x, y_w, y_l) = \sigma\left( \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} \right)$ weights examples where the policy still assigns high probability to the rejected reasoning chain.

Because the `Reasoning:` line comprises the majority of tokens in $y_w$ and $y_l$, the sum $\sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t})$ is heavily dominated by the `Reasoning:` tokens. By the chain rule:

- Every next-token prediction $y_{w,t}$ inside the valid `Reasoning:` trace receives a **positive gradient step**, increasing the probability that the model generates valid ambiguity analysis.
- Every next-token prediction $y_{l,t}$ inside the adversarial `neg_reasoning` trace receives a **negative gradient step**, actively suppressing the probability of generating flawed rationalizations.

## 3.3 3. Odds Ratio Preference Optimization (ORPO) — `run_orpo_training()`

### Input ($x$) vs. Contrastive Targets ($y_w, y_l$)

In `run_orpo_training()` (`trainer.py`, lines 394–488), ORPO consumes the exact same `dpo_train.jsonl` dataset and applies the identical `format_dpo()` transformation as DPO. Consequently, in an ML sense, the `Reasoning:` trace is used strictly as **part of the target output sequences $y_w$ (`chosen`) and $y_l$ (`rejected`)**.

### Monolithic Reference-Free Loss & Dual Gradient Update

Unlike DPO, ORPO is a single-stage, reference-free algorithm trained directly from the base model without an SFT warm-up or frozen $\pi_{\text{ref}}$. It achieves this by combining the supervised next-token Cross-Entropy loss on the chosen target $y_w$ with a log-odds ratio penalty between $y_w$ and $y_l$:

**ORPO Joint Objective Over Target Sequences $y_w$ and $y_l$**

$$\mathcal{L}_{\text{ORPO}}(\theta) = \mathbb{E}_{(x,y_w,y_l)}\Big[\underbrace{\mathcal{L}_{\text{SFT}}(x, y_w; \theta)}_{\text{Next-Token CE on } y_w} - \lambda \log \sigma\left(\log \frac{\text{odds}_\theta(y_w \mid x)}{\text{odds}_\theta(y_l \mid x)}\right)\Big] \tag{5}$$

where the length-normalized sequence probability $\bar{\pi}_\theta(y \mid x)$ and odds $\text{odds}_\theta(y \mid x)$ over the target tokens are defined as:

$$\bar{\pi}_\theta(y \mid x) = \exp\left(\frac{1}{|y|}\sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t})\right), \qquad \text{odds}_\theta(y \mid x) = \frac{\bar{\pi}_\theta(y \mid x)}{1 - \bar{\pi}_\theta(y \mid x)} \tag{6}$$

Thus, during every ORPO backward pass:

1. The valid `Reasoning:` tokens in $y_w$ receive **two simultaneous positive gradient signals**: one from $\mathcal{L}_{\text{SFT}}(x, y_w; \theta)$ (teaching the model the 4-field syntax and CoT rationale via next-token prediction) and a second from the odds-ratio term $\text{odds}_\theta(y_w \mid x)$.
2. The flawed `neg_reasoning` tokens in $y_l$ receive a **negative gradient signal** through $\text{odds}_\theta(y_l \mid x)$ in the denominator, penalizing the next-token probabilities of the incorrect reasoning chain.

## 3.4 4. Group Relative Policy Optimization (GRPO) — `run_grpo_training()`

GRPO (`trainer.py`, lines 490–621) operates under an online reinforcement learning paradigm that differs fundamentally from SFT, DPO, and ORPO. Understanding how `Reasoning:` works in GRPO requires separating **what happens to the dataset and reward calculation** from **what happens during on-policy rollout generation and policy gradient backpropagation**.

### Part A: Dataset Preparation & Programmatic Reward Calculation

1. **Pre-Generated Dataset `Reasoning:` Traces Are Discarded:** When `SingleTurnEnv._format_dataset()` (`src/ask_before_answer/rewards/base.py`, lines 67–98) loads `dpo_train.jsonl` for GRPO, it extracts only three fields: `prompt` ($x$), `target_action` (`"Clarify"` or `"Answer"`), and `target_response`. **The pre-generated `Reasoning:` text from the dataset is completely discarded and is never shown to the model or the reward functions.**
2. **Explicit Reward Evaluation Checks Only the `"Reasoning:"` Header:** When the policy generates a rollout completion $y_i$, the completion is graded by four programmatic criteria in `src/ask_before_answer/rewards/criteria.py`:
   - `FormatCriterion`: Checks boolean substring presence: `has_reasoning = "Reasoning:" in completion`.

It awards +1.0 if all four headers (`Action:`, `Reasoning:`, `Facets:`, `Response:`) are present, or applies a −2.0 penalty if any header is missing.

- `ActionCriterion`: Regex-matches `Action: (Clarify|Answer)` against `target_action`.
- `FacetLogicCriterion`: Parses `Facets: [...]` and verifies non-empty lists when `Action == Clarify` and empty lists when `Action == Answer`.
- `AccuracyCriterion`: Computes token-level F1 overlap between `Response:` and `target_response`.

Therefore, within the reward function code itself, **nothing evaluates the semantic prose written after `Reasoning:`**—only the presence of the `"Reasoning:"` tag is explicitly verified by `FormatCriterion`.

### Part B: Why the On-Policy `Reasoning:` Tokens Still Receive Gradients & Drive Learning

Even though the reward function only checks `"Reasoning:" in completion`, the tokens generated inside the `Reasoning:` line actively participate in learning through two core RL mechanisms:

1. **On-Policy Autoregressive Conditioning (Test-Time Compute):** For each prompt $x$, `GRPOTrainer` samples a group of $G = 8$ full completions $\{y_1, y_2, \ldots, y_8\} \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)$. In each rollout $y_i$, the model generates its own `Reasoning:` tokens on Line 2 *before* generating `Facets:` (Line 3) and `Response:` (Line 4). Through causal self-attention, a rollout $y_i$ that generates a coherent on-policy `Reasoning:` trace conditions the model to extract valid facets and produce a higher-accuracy response, thereby earning a higher total scalar reward $\mathcal{R}(x, y_i)$.
2. **Sequence-Level Advantage Broadcasting in the Policy Gradient:** GRPO normalizes the scalar rewards across the $G = 8$ rollouts to compute a scalar group advantage $A_i$ for each rollout $y_i$:

$$A_i = \frac{\mathcal{R}(x, y_i) - \text{mean}\big(\mathcal{R}(x, y_1), \ldots, \mathcal{R}(x, y_G)\big)}{\text{std}\big(\mathcal{R}(x, y_1), \ldots, \mathcal{R}(x, y_G)\big)} \tag{7}$$

Crucially, in token-level policy gradient optimization, **this single scalar advantage $A_i$ multiplies the log-probability ratio of every generated token $t \in \{1, \ldots, |y_i|\}$ in rollout $y_i$**—including every token in $\mathcal{T}_{\text{Reasoning}}$:

**GRPO Token-Level Policy Objective Showing Broadcast of Advantage $A_i$ to `Reasoning:` Tokens**

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\Big[\min\Big(r_{i,t}(\theta)A_i,\ \text{clip}\big(r_{i,t}(\theta), 1-\epsilon, 1+\epsilon\big)A_i\Big) - \beta\,\mathbb{D}_{\text{KL}}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)\Big] \tag{8}$$

where $r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$ is evaluated at every token position $t \in \mathcal{T}_{\text{Action}} \cup \mathcal{T}_{\text{Reasoning}} \cup \mathcal{T}_{\text{Facets}} \cup \mathcal{T}_{\text{Response}}$.

Because the sum over $t = 1 \ldots |y_i|$ spans the entire generated completion:

- When a rollout $y_i$ achieves above-average total reward ($A_i > 0$), backpropagation **increases the next-token probabilities of all `Reasoning:` tokens that rollout $y_i$ generated**.
- When a rollout $y_i$ achieves below-average total reward ($A_i < 0$), backpropagation **decreases the next-token probabilities of all `Reasoning:` tokens in rollout $y_i$**.

In short, GRPO treats the `Reasoning:` tokens as latent action steps in a trajectory: reasoning chains that lead to high-reward downstream facets and answers are reinforced, while reasoning chains that lead to errors are suppressed.

## 4. Comprehensive Summary Comparison Matrix

| Algorithm | Dataset `Reasoning:` Used? | ML Role in Forward Pass | How Loss & Gradients Act on `Reasoning:` Tokens | Codebase Reference |
|---|---|---|---|---|
| **SFT** | **Yes** (`output.reasoning` in `sft_train.jsonl`) | **Target** $y_t$ (for $t \in \mathcal{T}_{\text{Reasoning}}$) AND **Teacher-Forced Prefix** $y_{<t}$ (for downstream fields) | Direct token-level Cross-Entropy loss $-\sum_t \log \pi_\theta(y_t \mid x, y_{<t})$. Trains the model to emit the exact gold CoT tokens while conditioning `Facets:` and `Response:`. | `trainer.py` `run_sft_training()` (Lines 220–237) |
| **DPO** | **Yes** (`reasoning` in $y_w$; adversarial `neg_reasoning` in $y_l$) | **Contrastive Target Sequences** $y_w$ (`chosen`) and $y_l$ (`rejected`) | Sums next-token log-probs $\sum_t \log \pi_\theta(y_t \mid x, y_{<t})$ across all completion tokens. Pushes **positive gradients** onto valid `Reasoning:` tokens in $y_w$ and **negative gradients** onto flawed `neg_reasoning` tokens in $y_l$ relative to $\pi_{\text{ref}}$. | `trainer.py` `run_dpo_training()` (Lines 318–344) `preprocess.py` (Lines 452–520) |
| **ORPO** | **Yes** (`reasoning` in $y_w$; adversarial `neg_reasoning` in $y_l$) | **Contrastive Target Sequences** $y_w$ (`chosen`) and $y_l$ (`rejected`) | Reference-free joint loss $\mathcal{L}_{\text{SFT}}(y_w) + \lambda \mathcal{L}_{\text{OR}}(y_w, y_l)$. Valid `Reasoning:` tokens in $y_w$ receive **both** SFT Cross-Entropy gradients and positive log-odds gradients; flawed `neg_reasoning` tokens in $y_l$ receive negative log-odds gradients. | `trainer.py` `run_orpo_training()` (Lines 415–441) |
| **GRPO** | **No** (Dataset CoT is discarded in `SingleTurnEnv`; model generates its own CoT on-policy) | **On-Policy Sampled Rollout Tokens** $y_{i,t} \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)$ | **Reward Check:** `FormatCriterion` only checks `"Reasoning:" in completion`. **Policy Gradient:** The rollout's total advantage $A_i$ multiplies $\nabla_\theta \log \pi_\theta(y_{i,t} \mid x, y_{i,<t})$ for *every* token $t$ in rollout $y_i$, reinforcing on-policy reasoning traces that lead to high-reward facets and answers. | `trainer.py` `run_grpo_training()` (Lines 490–621) `rewards/base.py` (Lines 67–98) `rewards/criteria.py` (Lines 17–28) |